

🔗 TECHNICAL SYSTEM SPECIFICATION

AI FAQ Assistant API

Enterprise RESTful backend service engineered with Node.js, Express, MongoDB, and integrated Google Gemini AI context synthesis.

📄 Node.js & Express

🗄️ MongoDB Mongoose

🔗 Google Gemini AI

🔑 JWT Authorization



Executive Summary



Core Mission

Automates FAQ generation and centralizes technical documentation workflows through Gemini AI models.



Decoupled Stack

Node.js and Express backend fully isolated from client applications via standard REST interfaces.



Role Security

JWT bearer token authentication guarantees strict role authorization across private endpoints.

📈 Performance Impact: Drastically cuts support response latency while enforcing strict Mongoose schema validation.

Case Study: Support Desk Transformation

01. Context

Manual documentation and high support ticket backlogs.

02. Challenge

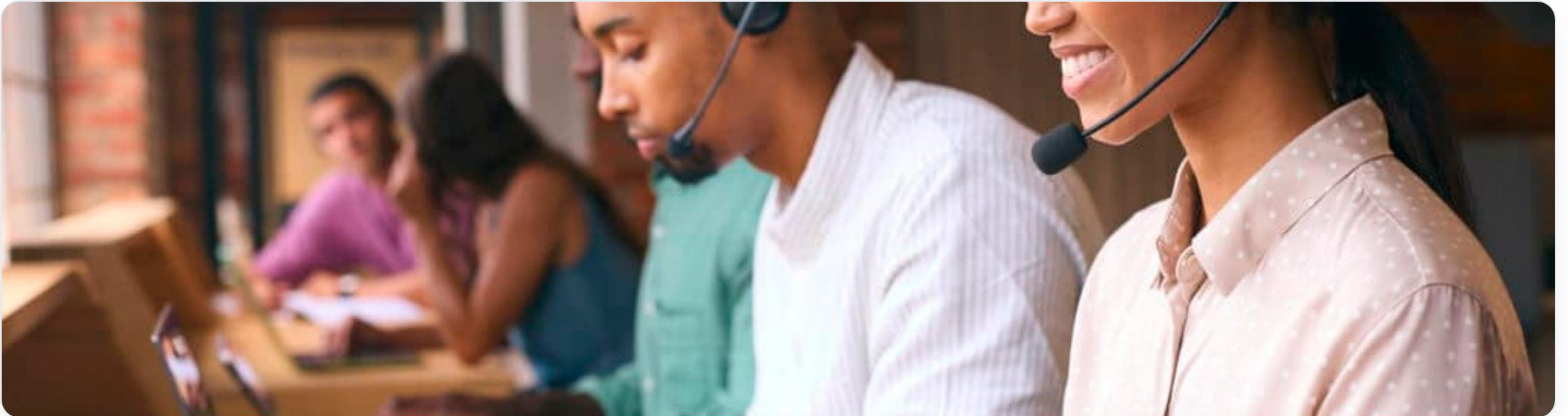
Unindexed queries, manual entry, and unverified user roles.

03. AI Solution

Google Gemini synthesis with JWT authorization flows.

04. Outcome

Instant answer synthesis and verified author auditing.



Tech Stack & Hardware Requirements

Software Component Matrix

LAYER	TECHNOLOGY SPECIFICATION
Runtime Environment	Node.js (v18.0+ LTS)
Web Framework	Express.js (v4 / v5)
Database & ODM	MongoDB & Mongoose ODM
AI Engine Integration	Google Gemini SDK (@google/genai)

Hardware Thresholds

-  **CPU Architecture:**
Intel Core i5 (8th Gen) or AMD Ryzen 5 minimum.
-  **System Memory:**
8 GB DDR4 (16 GB recommended for local DB).
-  **Disk Storage:**
1 GB available SSD workspace.

5-Layer Architecture Pattern

 **1. API GATEWAY** Express.js handles CORS, parses JSON payloads, and directs REST routing.

 **2. MIDDLEWARE** Validates JWT bearer tokens, verifies user permissions, and sanitizes incoming data.

 **3. CONTROLLER** Executes controller business logic and formats standardized JSON API responses.

 **4. SERVICE LAYER** Integrates Google Gemini SDK for dynamic prompt execution and AI response synthesis.

 **5. PERSISTENCE** MongoDB collections managed via Mongoose schemas for indexing and data integrity.

Mongoose Schemas & Model Structure



User Entity

Stores account credentials: `name`, `email` (Unique), `password` (Bcrypt Hashed), and `role` claims.



FAQ Entity

Stores `question`, `answer`, `category` enum, and relational ObjectId reference to `createdBy` User.



Category Schema

Restricts domain classifications: Technology, Education, Health, Banking, and General support topics.

Role-Based Access Control (RBAC)

System Role	Authentication Level	Permitted System Capabilities
 Admin	JWT Required	Full control: user management, edit/delete any FAQ, inspect system logs.
 Content Creator	JWT Required	Create FAQs, update/delete owned records, trigger AI generation tool.
 Authenticated User	JWT Required	View public & internal FAQs, execute semantic search, submit feedback.
 Public Reader	None (Public)	Read public FAQs and execute basic keyword search queries.

Project Structure & Dependencies

Project File Tree

```
FAQ/  
├── package.json  
├── .env  
└── src/  
    ├── app.js  
    ├── server.js  
    ├── config/db.js  
    ├── controllers/  
    │   ├── authController.js  
    │   └── faqController.js  
    ├── middleware/  
    │   └── authMiddleware.js  
    └── models/  
        ├── FAQ.js  
        └── User.js
```

Package Initialization

```
npm init -y
```

Core Dependencies

```
npm install express mongoose dotenv cors jsonwebtoken bcrypt @google/genai
```

SOURCE CODE

Core Server & Middleware Implementation

// src/server.js

```
const app = require('./app');
const connectDB = require('./config/db');

const startServer = async () => {
  await connectDB();
  const PORT = process.env.PORT || 5000;
  app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
  });
};

startServer();
```

// src/middleware/authMiddleware.js

```
const jwt = require('jsonwebtoken');

const protect = async (req, res, next) => {
  let token = req.headers.authorization?.split(' ')[1];
  if (!token) {
    return res.status(401).json({ message: 'No token' });
  }
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  req.user = await User.findById(decoded.id).select('-password');
  next();
};
```

API Endpoint Specification Matrix

Method	Endpoint Path	Access Level	Description
POST	/api/auth/register	Public	Registers a new user account and returns JWT session token.
POST	/api/auth/login	Public	Authenticates user credentials and yields authorization bearer token.
POST	/api/faqs	Private (Bearer)	Creates new FAQ record associated with authenticated user ID.
GET	/api/faqs/search	Public	Executes keyword search query using ?q= URL parameters.
POST	/api/ai/generate-faq	Private (Bearer)	Invokes Gemini AI model to auto-synthesize context QA pairs.

VERIFICATION

Payload Testing & Execution Verification

```
// POST /api/faqs Request Payload
```

```
{  
  "question": "What is Node.js?",  
  "answer": "Node.js is an open-source JS runtime.",  
  "category": "Technology"  
}
```

```
// HTTP 201 Created Response
```

```
{  
  "success": true,  
  "data": {  
    "_id": "6a3e667f21edd9fd9b704ab1",  
    "question": "What is Node.js?",  
    "createdBy": "6a3e64d321edd9fd9b704aaa"  
  }  
}
```

✓ PROJECT DELIVERY COMPLETE

Project Execution & System Verification



API Testing Demonstration

Complete walkthrough of Postman collection endpoints and response status assertions.



Interactive Runtime Demo

Live application execution showcasing Google Gemini AI integration and JWT session flows.

AI FAQ Assistant API v1.0

 MERN Architecture

Image Sources



https://images.stockcake.com/public/a/7/1/a7132125-dcf1-49e0-b242-5369620ae1cb_large/digital-brain-concept-stockcake.jpg

Source: stockcake.com



Thumbnail for <https://citypersonnel.net/wp-content/uploads/2024/10/multi-cultural-customer-support-or-telesales-team-2024-10-19-15-04-56-utc-1024x1024.jpg>

citypersonnel.net Source: citypersonnel.net